KAREL

Werkblad 3: Algoritmen om problemen op te lossen

Naam: _____

Tijd	Wat gaat er
0:00–0:20	Uitwerking huiswerk + uitleg loops
0:20–0:45	Programmeren zonder computer
0:45–1:00	Pauze
1:00–1:45	Programmeren zonder computer
1:45–2:00	Pauze
2:00–2:35	Programmeren met computer
2:35–2:45	Toelichting tentamen en uitdelen huiswerk

1 Loops Cheat Sheet

An `IF` lets Karel do something *once, or not at all.* Two kinds of loop let Karel do something *again and again*: `ITERATE` repeats a fixed, known number of times; `WHILE` repeats for as long as a test keeps holding, however many times that turns out to be.

Concept

```
ITERATE n TIMES
  <instruction>
```

Karel repeats the instruction exactly `n` times, where `n` is a fixed number you already know when you write the program.

Concept

```
WHILE <test> DO
  <instruction>
```

Karel checks the test *first.* If it holds, he runs the instruction and then checks again until the test finally fails. If the test is already false the very first time, the instruction runs **zero** times. That is a feature, not a bug: a well-written `WHILE` is always safe to call, even when there is nothing to do.

Choosing a loop. Use `ITERATE` when you know the count before the program runs: climb exactly 3 steps, plant exactly 4 leaves. Use `WHILE` when you don't know the count in advance, but you do know a condition that means "keep going", for example walk until a wall stops you, however far away that turns out to be.

The pattern you will use all day. Almost every loop is some version of "keep going until a wall stops you":

```
WHILE front-is-clear DO
  move
```

Karel does not need to know *how far* the wall is. He just walks until he gets there. That is the whole point of a loop: you describe the *condition to stop*, not the number of steps.

Here's your current toolbox

You may use all of the following instructions freely, without redefining them. When testing with a computer, you *will* need to write them!

Basic	`move`, `turnleft`, `pickbeeper`, `putbeeper`, `turnoff`
Turning	`turnright`, `turnaround`
Facing	`face-north`, `face-east`, `face-south`, `face-west`

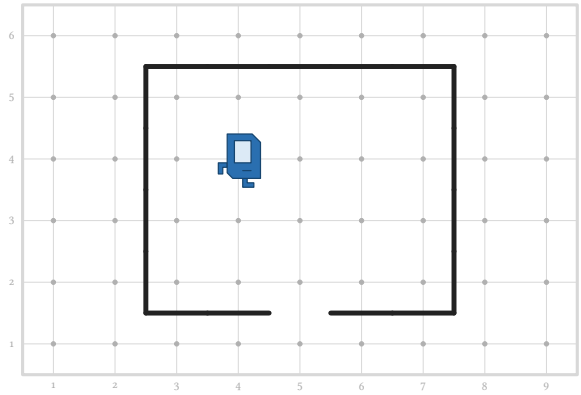
Code Quality

The tasks are bigger now, so *how* you write matters as much as *whether* it works. Four habits:

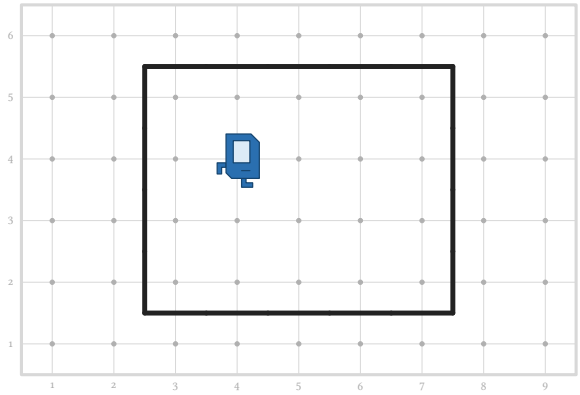
- **Name what repeats.** If the same lines appear twice, give them a name and call it twice instead.
- **Reuse what you have.** Use instructions from the toolbox before writing new code.
- **Loop, don't copy.** Two identical moves might be fine; twenty are a loop you forgot to write.
- **Simplify the `IF`s.** The four transformations from two weeks ago (test reversal, top / bottom factoring, redundant-test) still apply.

2 Escape the Room

Karel wakes up inside a rectangular room built of walls. Somewhere there *might* be a single doorway, represented as a gap in the wall. If there is, Karel must walk out through it. If the room is sealed shut, Karel must give up and `turnoff`.



A room with a doorway (south side, avenue 5). (`escape1.w`)



A sealed room. Here Karel must also `turnoff` at some point! (`escape2.w`)

The hard part here is **knowing when to stop**. If Karel just follows the wall looking for a gap, how does he ever find out that there *is* no gap? He would circle around forever. You are allowed one beeper to solve this.

↪ **The idea.** Karel starts in the middle of the room, not against a wall, so don't drop the breadcrumb yet: first walk straight ahead until you actually reach a wall. *There*, on the first corner where the way ahead is blocked, drop a beeper as your breadcrumb. Then follow the wall around the inside of the room, checking at each corner whether you can step out. If you ever arrive back on the breadcrumb, you have gone all the way around and found no door. That is when Karel can safely `turnoff`.

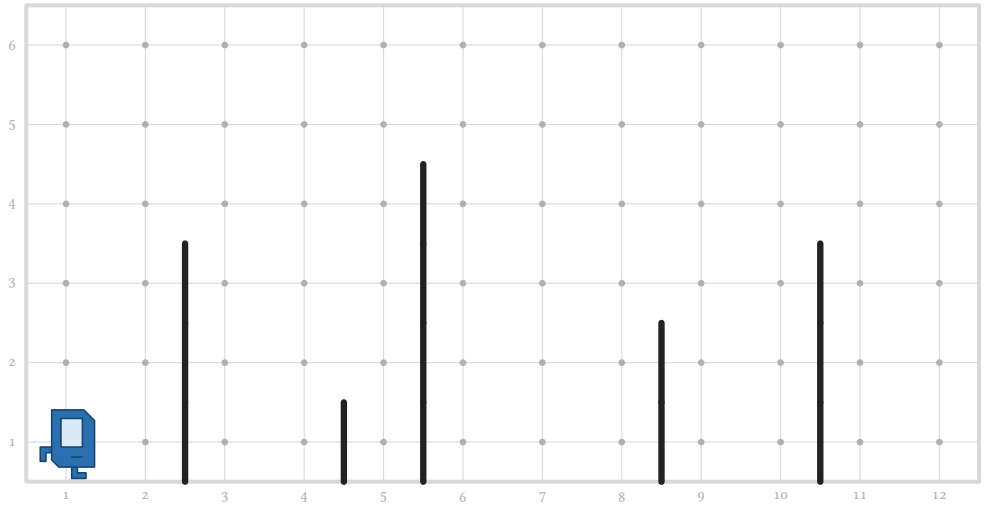
↪ **Plan your instruction hierarchy.** List the new instructions you will define and what each is responsible for.

Instruction name	**What it is responsible for**

↪ **What will the main code look like?**

3 Super Steeplechase

Karel is doing some sports today: hurdling. He needs to go from West all the way to East, while "clearing hurdles", so going over them and step on the ground after each. The trajectory is pretty tough: the hurdles sit at **arbitrary positions** and are of **arbitrary height**, with no known upper bound. Here's an example:



`steeple2.w`

That means that Karel has to keep going upward for as long as is needed, go around the hurdle, then descend until he hits the ground. But not being able to count, he really really needs his `WHILE` loops! Repeating until he's there where he needs to be, either up, down, or when sprinting to the next hurdle.

↪ **The shape of the solution.** Let's say that there are two main things Karel does, alternating between them: **run** along the ground until a hurdle blocks the way, and **jump** the hurdle he just met: first going up the near side while the wall is still beside his, then one step across the top, and finally going down the far side to the ground. Notice that "up the side while the wall is beside me" is already a `WHILE`, letting Karel handle a hurdle of any height.

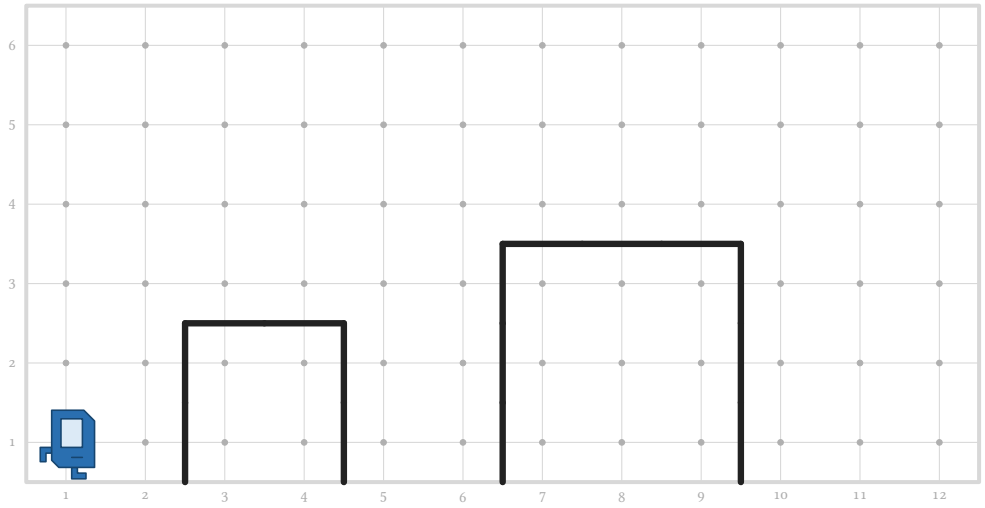
↪ **Plan your instruction hierarchy**. Clearly you have `run` and `jump`, but the jump can be separated in parts, too.

Instruction name	What it is responsible for

↪ **What will the main code look like?**

4 Superduper Steeplechase

Now the hurdles have **width**: each one is a solid block, so its top is a little plateau Karel must walk *across* before it can come down the far side.



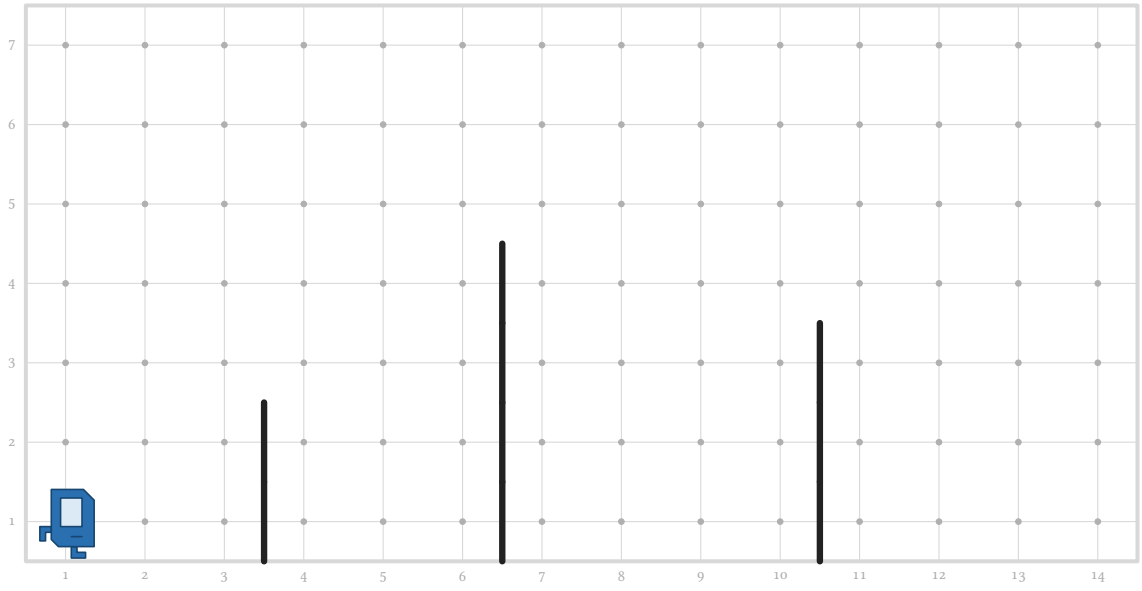
steeple3.w

Your `jump` from the Super Steeplechase almost works, but needs an extra part to walk across the top of the hurdle.

↬ **Adapt your previous program** so the same code clears both the thin hurdles and the wide ones! Write your changed `jump` here, plus an additional instruction that `jump` uses:

5 Spring Leaves

You have seen this assignment in Week 1: a row of trees, each a bare trunk that needs a crown of leaves on top. Back then every trunk was the same height and evenly spaced, perfect for "hardcoding" but not very flexible. This year, spring is unruly: the trunks are of **random height** at **random positions**. Karel must still crown every one. A finished crown is **four leaves (a beeper) surrounding the top of the trunk**.



`leaves2.w`

↬ **You have already written most of this.** Crowning a trunk is the *same journey* as jumping a hurdle! Reuse your steeplechase design, adding the leaves, and ideally, make it better.

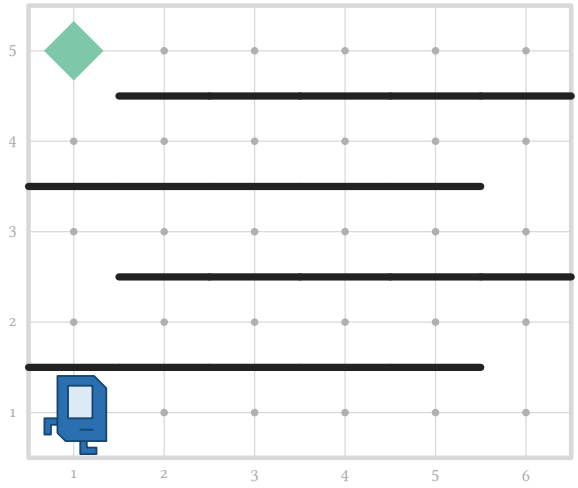
↬ **Plan and build.** List your instructions, then run it until every trunk wears its four leaves.

Instruction name	What it is responsible for

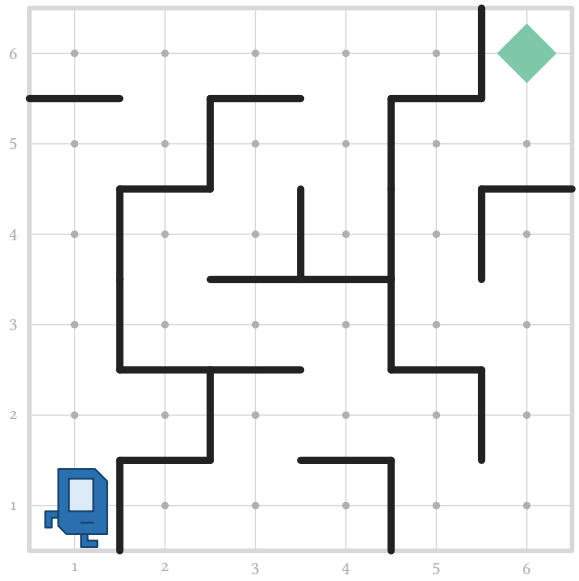
↬ **What will the main code look like?**

6 Maze

Karel is dropped at (1,1) facing East inside a maze and must find the single beeper hidden in it, then pick it up and `turnoff`. Karel has no map, but there is an old trick that needs none: **keep your right hand on the wall and walk**. Whoa, you know that one! In a maze like this one you will eventually reach every corner, including the one with the beeper. Perfect.



`maze1.w`



`maze2.w`

↬ **The wall-follower you built.** For the homework you wrote `followRightWall` — the right-hand rule that takes a single step along the wall: take the opening on your right if there is one, otherwise go straight, otherwise turn left. That one instruction is the whole secret of the maze. Get your definition out and keep it in front of you.

↬ **Write the main loop.** `followRightWall` takes exactly one step of the journey. What loop makes Karel keep taking steps *until* he is standing on the beeper? Write the full `BEGINNING-OF-EXECUTION`, ending by picking the beeper up and turning off.

```
BEGINNING-OF-EXECUTION
```

7 On the computer

Have the teacher or assistant check your work first.

↬ **Build Escape the Room and see if it works for both worlds.** It must walk out of the first and turn off in the second, with no change to the code in between. Anything that's off?

↬ **Build Superduper Steeplechase and see if it works.** Anything that's off?

↬ **Build Spring Leaves and see if it works.** Anything that's off?

↬ **Build your Maze solver and watch it run.** Then try moving the beeper to a different corner without touching your program. You can edit the world file by opening it and changing the text! Does it still get there?

↬ **Left hand instead of right.** Rewrite `follow-right-wall` as `follow-left-wall`, following the wall on the other side. Change as little as possible — and say, in one sentence, exactly what you changed and why that is all it took.